C Details on Internal Representations Analysis

C.1 Raw Similarity Scores for Benchmarking

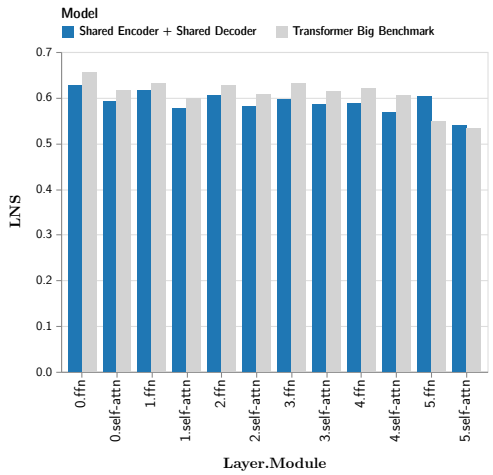
We establish a *benchmark* score for the expected similarity of our two metrics by comparing the baseline Transformer Big with identical models trained from different random seeds. Table 9 presents the raw similarity scores from which we compute the normalized scores presented in Table 5. As shown, the similarity between

Architecture	Encoder		Decoder	
	CKA	LNS	CKA	LNS
`TransformerBig` Seed 2	.96	.61	.94	.62
`TransformerBig` Seed 3	.96	.62	.95	.62
`SharedEnc`	.94	.58	.95	.62
`SharedDec`	.97	.62	.93	.59
`SharedEncSharedDec`	.95	.59	.94	.59
`SharedEncDec`	.94	.57	.93	.58
`NoEnc`	.87	.43	.95	.60
`NoDec`	.96	.60	.90	.54
`ShareEncNoDec`	.94	.59	.92	.55
`ShareEncNoDec`$^{d'_{\text{ff}}=41952}$	.94	.51	.89	.51

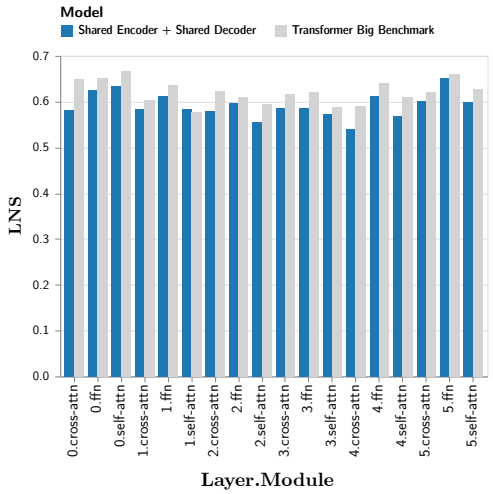
Table 9: Raw similarity of the representations of corresponding layer-modules of different architectures vs. the Transformer Big for WMT22 EN →DE. For *NoDec* configurations we compare the final output of the transformer layer as a whole as they have different submodules. The columns for shared and for dropped FFNs are highlighted in gray and blue respectively.

C.2 Layer-wise Analysis

In Table 5, we report the aggregated similarity scores across all layers of Transformer encoder and decoder. Here, we report a more fine-grained layer-wise similarity score mostly to showcase the reliability of the aggregated scores. In Figure 2, we plot layerwise LNS to study how similar the semantic information captured at each layer is to that of the baseline model at *every layer*. When LNS scores are high, the network is producing similar local neighborhoods for each sentence in our evaluation set. In particular, we are interested in comparing the benchmark LNS scores and those of `SharedEncSharedDec` at each layer. As shown, the layer-wise LNS scores of `SharedEncSharedDec` track the baseline scores at almost every layer, confirming the reliability of the aggregated score. We observe similar pattern for all the models that we evaluate in this paper.



(a) Encoder



(b) decoder

Figure 2: Layerwise LNS between `SharedEncSharedDec` and `Transformer Big` (blue bars). LNS between two versions of Transformer Big trained from different random initializations are shown by the grey bars to ground the comparison. FFN sharing does not dramatically change activations produced at each layer.

D Effect of batch size on decoding speed

In Section 4.3, we compared the decoding speeds of the One Wide FFN model and the Transformer Big, with a batch size of 1. In Table 10, we delve into how the decoding speed evolves as the batch size increases. As shown, the One Wide FFN model is faster for smaller batch sizes, but its advantage diminishes as the batch size increases, being slower than the Transformer Big for large batch sizes. We suspect this slowdown is due to the fact that the

Batch	Transformer Big	*One Wide FFN*	Speed-up (%)	*# batches*
1	$110.8^{\pm 1.2}$	$137.5^{\pm 1.1}$	24	$2,047$
2	$221.7^{\pm 14.3}$	$260.9^{\pm 6.5}$	18	$1,024$
4	$397.4^{\pm 8.0}$	$448.9^{\pm 2.0}$	13	512
8	$718.3^{\pm 8.0}$	$748.7^{\pm 10.6}$	4	256
16	$1,220.7^{\pm 56.2}$	$1,226.9^{\pm 17.2}$	1	128
32	$1,958.5^{\pm 112.4}$	$1,837.6^{\pm 15.3}$	-6	64
64	$1,319.1^{\pm 36.7}$	$1,259.0^{\pm 70.0}$	-5	32
128	$1,925.1^{\pm 64.8}$	$1,705.0^{\pm 62.3}$	-11	16
256	$2,312.1^{\pm 67.4}$	$1,976.5^{\pm 123.2}$	-15	8
512	$2,512.0^{\pm 50.1}$	$1,957.9^{\pm 32.6}$	-22	4

Table 10: Effect of batch size on decoding speed (in tokens/s) for the Transformer Big and *One Wide FFN* ($d_{\text{ff}'} = 49,152$). Δ is the percentage change in inference speed, and # batches is the number of batches used to evaluate. For large batch sizes, there are fewer batches (since the dataset size is fixed), which leads to higher variance in the measurements.

large FFN size requires higher peak memory, making the larger sizes non-optimal for this model.